

РЕЦЕНЗІЯ

на кваліфікаційну роботу бакалавра
студента 4-го курсу, групи 444(Б)

Мулика Андрія Васильовича

Спеціальність 122 «Комп'ютерні науки»

навчально-науковий інститут фізико-технічних та комп'ютерних наук

Чернівецький національний університет імені Юрія Федьковича

на тему: « Розробка самовідновлювального конвеєру ETL та аналітичного сховища для аналізу текстових відгуків з використанням Apache Airflow і локальних великих мовних моделей »

Тема роботи є актуальною: поєднання механізмів самовідновлення даних, локального LLM-інференсу та веб-орієнтованого моніторингу в єдиному ETL-конвеєрі є практично значущим і відповідає сучасним тенденціям DataOps та MLOps. Студент розробив самовідновлювальний конвеєр на базі Apache Airflow з детерміністичним модулем виявлення та усунення п'яти класів дефектів вхідних даних, аналізом тональності через локальну модель llama3.2 (Ollama REST API), колонковим аналітичним сховищем DuckDB та Blazor Hosted WebAssembly дашбордом для моніторингу запусків.

Сильні сторони роботи

Архітектурна повнота та інженерна зрілість системи. Студент реалізував повноцінну багатокомпонентну систему з чітким розподілом на три незалежні рівні: обробка (Python/Airflow), зберігання (DuckDB + SQLite + JSON) та відображення (ASP.NET Core + Blazor WASM). Вибір DuckDB як вбудованого OLAP-сховища підкріплено кількісними вимірюваннями: аналітичні SQL-запити виконуються у 300–500 разів швидше порівняно з прямим скануванням JSON-файлів, що є доведеним результатом, а не декларацією. Архітектурне рішення про дворівневе зберігання (DuckDB для вихідного датасету, JSON-файли для результатів конвеєру) забезпечує незалежність між підсистемами без введення зайвих залежностей.

Якість реалізації модуля самовідновлення та graceful degradation. Модуль самовідновлення реалізує патерн «Ланцюжок відповідальності» з детерміністичною обробкою п'яти класів дефектів і пріоритезованою черговістю перевірок. Принципово важливим є збереження `original_text` поряд з `healed_text` для кожного виправленого запису — це забезпечує повний аудиторський слід і відображається у Healing Report дашборду. Функція `_degraded_results` реалізує Null Object патерн: при повній недоступності Ollama конвеєр завершується успішно зі статусом DEGRADED або CRITICAL замість аварійного завершення, що є коректною реалізацією принципу graceful degradation.

Безпека автентифікації та якість реалізації. Власний Bearer-токен механізм реалізовано з PBKDF2-SHA256 хешуванням паролів (100 000 ітерацій, 32-байтна криптографічна сіль) та токенами з ~384-бітною ентропією — що відповідає рекомендаціям OWASP Password Storage Cheat Sheet. Паралельне завантаження восьми аналітичних ендпоінтів через `Task.WhenAll()` скорочує час відповіді до максимуму з восьми запитів замість їх суми. Real-time моніторинг DAG через `PeriodicTimer` з парсингом логів Airflow через регулярний вираз є нетривіальним і практично виправданим рішенням.

Комплексне тестування з кількісними результатами. Четвертий розділ містить реєстр із п'ятнадцяти тест-кейсів чотирьох рівнів (модульний, інтеграційний, системний, функціональний), що рідко зустрічається у бакалаврських роботах. Кількісний аналіз продуктивності (таблиці 4.6 і 4.7) дозволяє точно ідентифікувати вузьке місце системи:

послідовний LLM-інференс на CPU (~5 с/відгук) займає 96–98% загального часу виконання DAG. Виявлена кореляція між зірковим рейтингом та передбаченою тональністю (таблиця 4.4) підтверджена з кількісним розподілом і узгоджується з академічними джерелами.

Технологічний стек та його відповідність задачі

Обраний стек повністю відповідає вимогам задачі.

Apache Airflow 3.0 + TaskFlow API. Використання TaskFlow API (декоратори `@dag`, `@task`) є рекомендованим підходом для Python-нативних конвеєрів у Airflow 2.0+: залежності між задачами стають явними через сигнатуру функцій без `XCom.push()/pull()`. Параметризація через Param API з типізованими параметрами (`input_file`, `batch_size`, `offset`, `ollama_model`) забезпечує конфігурованість запусків безпосередньо з Blazor-дашборду через REST API Airflow.

DuckDB + DuckDB.NET. Застосування нативної функції `read_json_auto()` для прямого INSERT без проміжної десеріалізації в C#-об'єкти є технічно грамотним рішенням. Колонкове зберігання з `vectorized execution` і `zone map`-фільтрацією забезпечує субсекундні відповіді на 6+ ГБ датасеті, що підтверджується вимірними показниками у таблиці 4.7.

Ollama + llama3.2 + промпт-інжиніринг. Архітектурне рішення розгортати Ollama на хост-машині та отримувати до неї доступ з Docker-контейнера через `host.docker.internal:11434` дозволяє ефективно використовувати GPU хосту без прокидання GPU у контейнер. Треступеневий парсинг відповіді (видалення markdown-обгортки → JSON-десеріалізація → fallback пошук ключових слів) є практично обґрунтованим рішенням для нестабільних LLM-відповідей.

Зауваження

1. Продуктивність LLM-інференсу є критичним системним обмеженням, яке залишається невирішеним в межах роботи. Аналіз показує, що `analyze_sentiment` займає 96–98% часу виконання DAG (~5 с/відгук на CPU), а обробка 1000 відгуків потребує ~1,4 години. При цьому автор лише констатує це як «вузьке місце» і згадує GPU-прискорення як перспективу, не реалізуючи жодного конкретного рішення в межах роботи. Відсутній навіть базовий паралелізм запитів до Ollama через `asyncio.gather()` або `ThreadPoolExecutor`, реалізація якого не потребувала б зміни архітектури. Система, де аналіз 1000 відгуків займає 83 хвилини при заявленій орієнтації на «виробниче» використання, є практично непридатною для реальних навантажень без цього покращення.
2. Механізм автентифікації потребує доопрацювання в частині зберігання токенів. Bearer-токени зберігаються у `localStorage`, тоді як рекомендованою практикою є використання `httpOnly cookie`. Автор коректно застосував PBKDF2-SHA256 на стороні сервера, однак клієнтська частина потребує відповідного вдосконалення.

Загальний висновок

Кваліфікаційна робота виконана на високому інженерному рівні та демонструє практичне розв'язання нетривіального технічного завдання: поєднання детерміністичного `self-healing`, локального LLM-інференсу, колонкового аналітичного сховища та повнофункціонального Blazor-дашборду в єдиній системі. Студент продемонстрував впевнене володіння стеком Python/Airflow, DuckDB, ASP.NET Core/Blazor та принципами безпечної автентифікації. Зазначені зауваження стосуються критичного обмеження продуктивності, яке залишилось без практичного вирішення в межах роботи, та свідомо

залишеної вразливості безпеки, що суперечить позиціонуванню системи як виробничого зразка. Кваліфікаційна робота відповідає вимогам до робіт освітнього рівня «бакалавр» за спеціальністю 122 «Комп'ютерні науки» та заслуговує на оцінку **«добре»**.

Academy Program Lead,
ТОВ "Українські інформаційні технології",
кандидат економічних наук



Вінничук І.С.